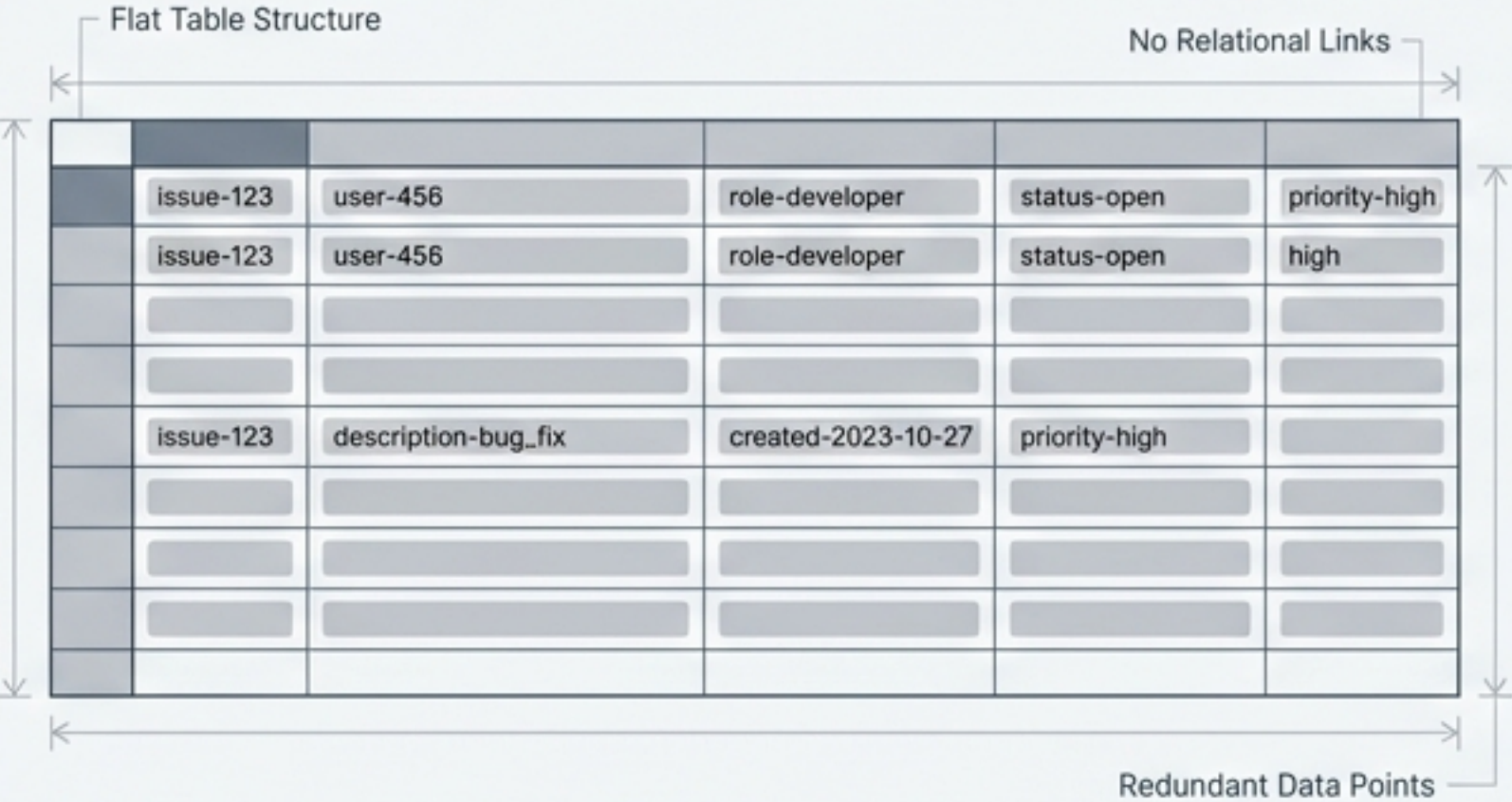


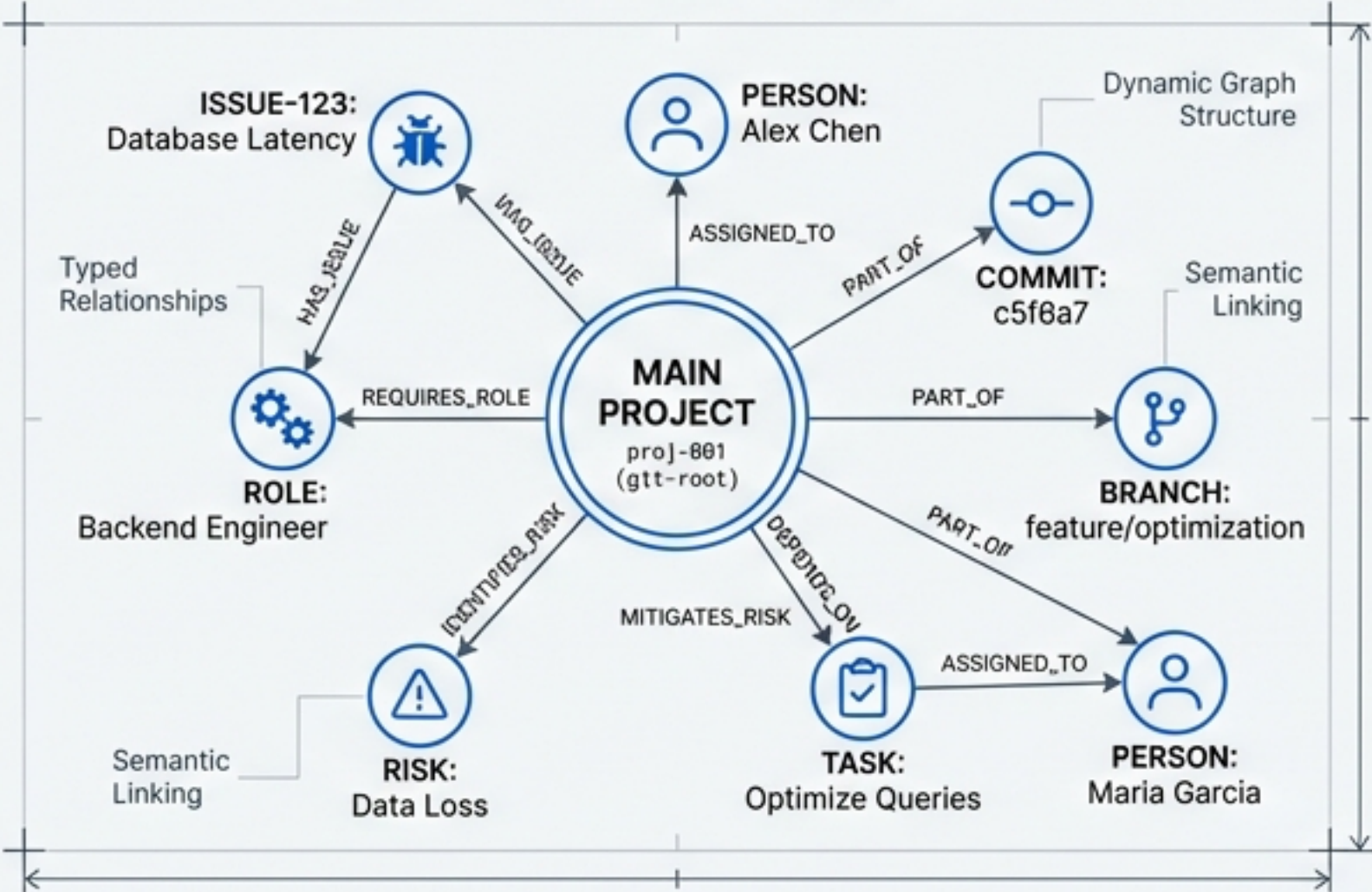
Shifting the Source of Truth to a Git-Native Graph

We are transitioning from a file-based linear structure to a graph-based node and edge system. This architecture treats every entity—issues, people, and roles—as unique nodes connected by typed relationships.

v0.2.2: Isolated Records



v0.2.4: Connected Ecosystem



Key Benefits



Visualisation

Enable powerful dependency mapping and querying.



Coordination

Designed explicitly for AI agent interaction.



Persistence

Maintains Git as the absolute source of truth.

The Limitations of Linear Lists

Traditional issue tracking treats tickets as isolated records, creating friction points for modern development.

1. No Semantic Context

Issue-756  is opaque. Without digging, we cannot determine if it is a bug, a feature, or who owns it.

2. Weak Relationships

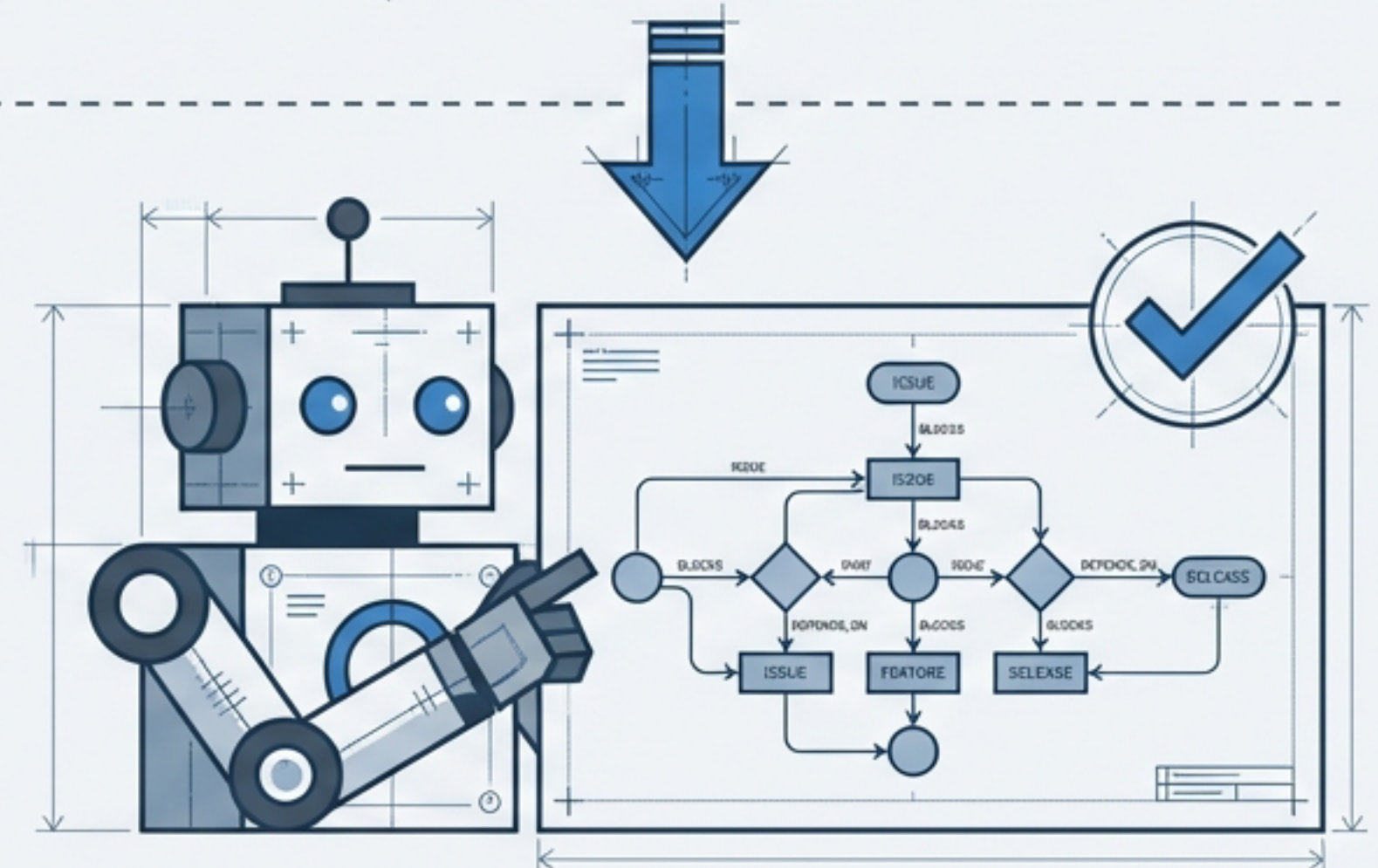
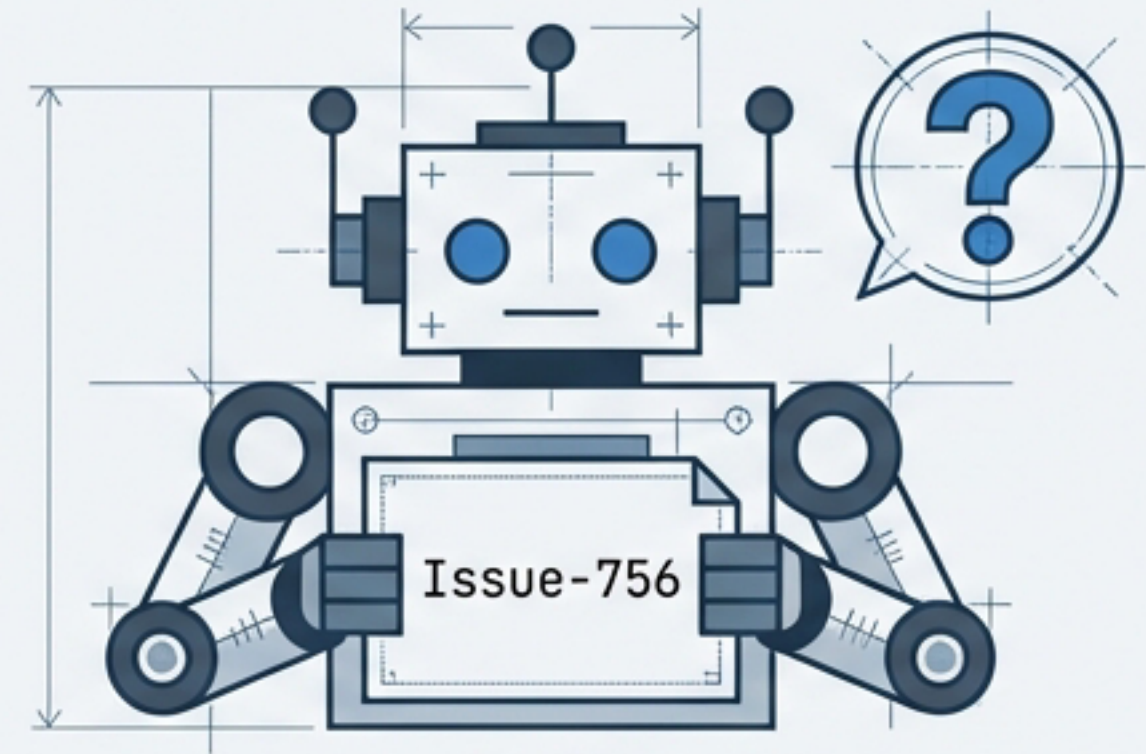
Issues exist in isolation. We cannot easily answer complex queries like "What bugs block this release?"

3. Poor Agent Coordination

AI agents struggle with flat lists. They require structured context about dependencies and workflow states.

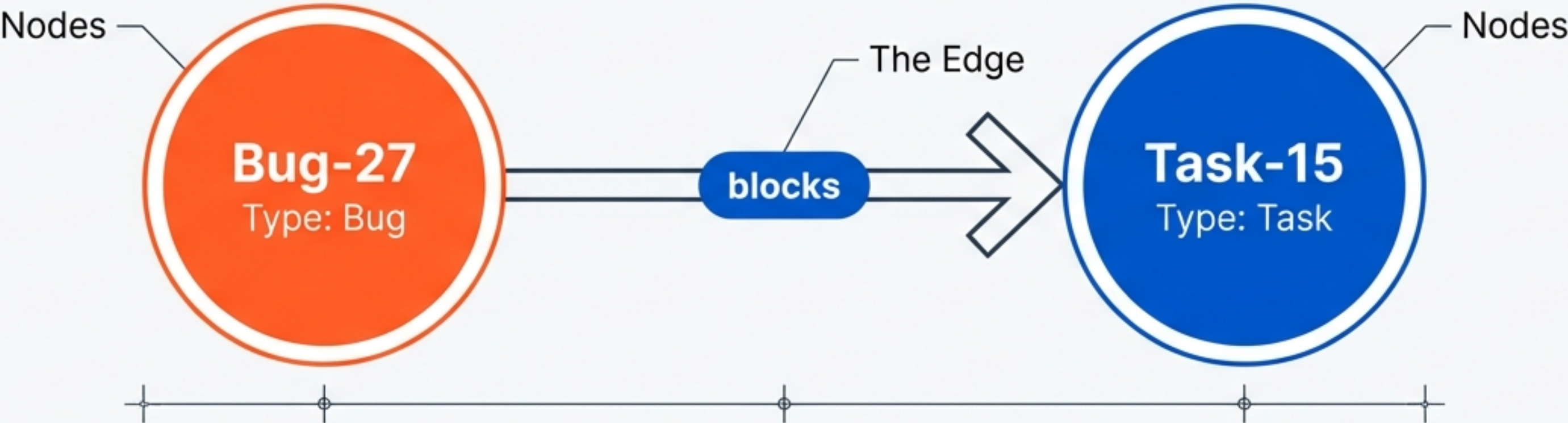
4. Limited Visualisation

Unstructured data prevents the generation of meaningful dependency maps or critical path views.



The Graph Paradigm: Nodes & Edges

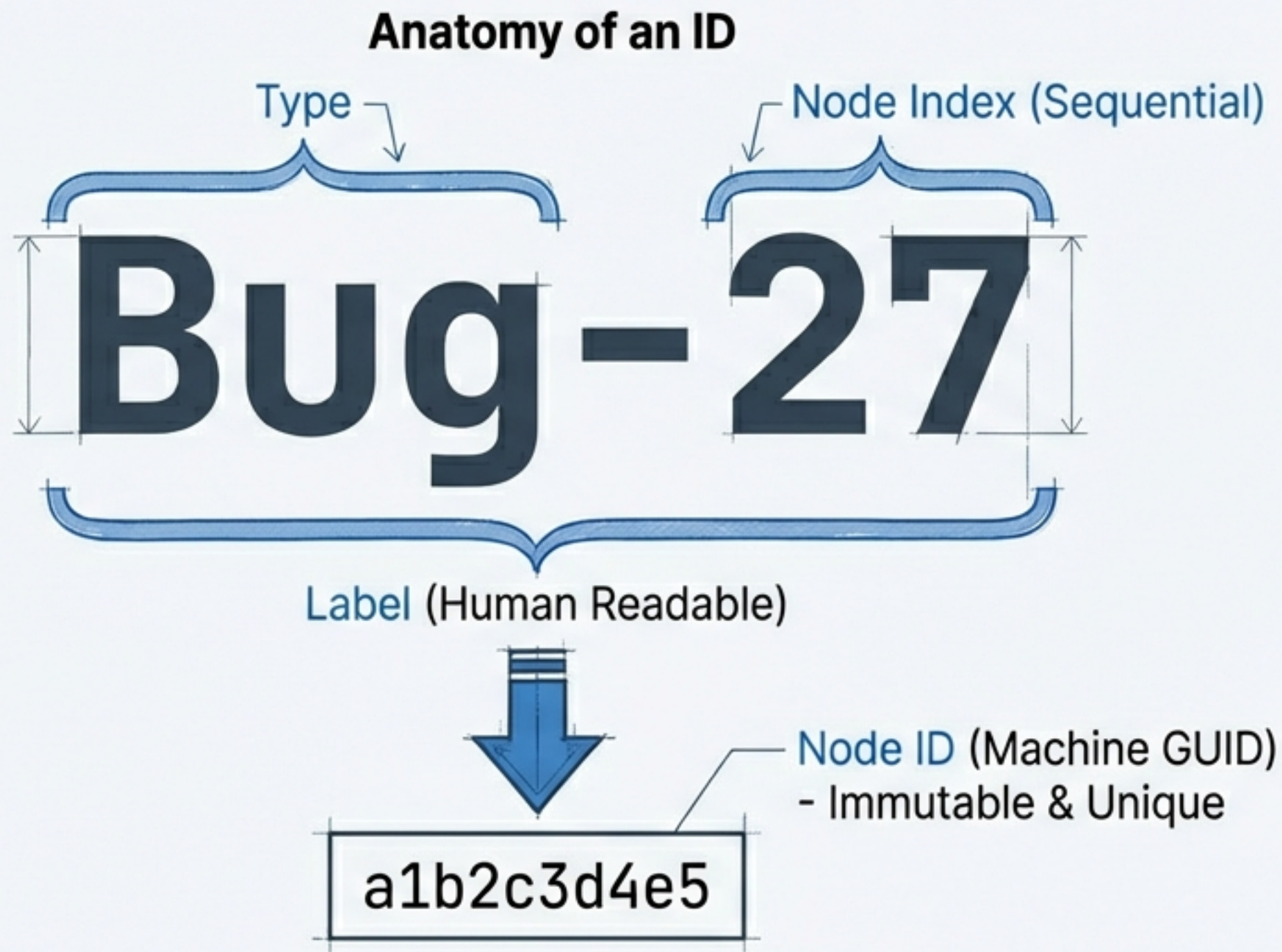
By treating issues as nodes in a graph, we unlock rich identity and explicit relationships.



	Concept	Description	Example
1	Node	Any entity in the system	Bug, Task, Feature, Person
2	Edge	The relationship connecting two nodes	blocks, assigned-to
3	Link Type	The semantic definition of that relationship	{ verb: 'blocks', inverse: 'blocked-by' }

A Hybrid Identity Model

Every node requires an identity that is machine-precise yet human-readable.



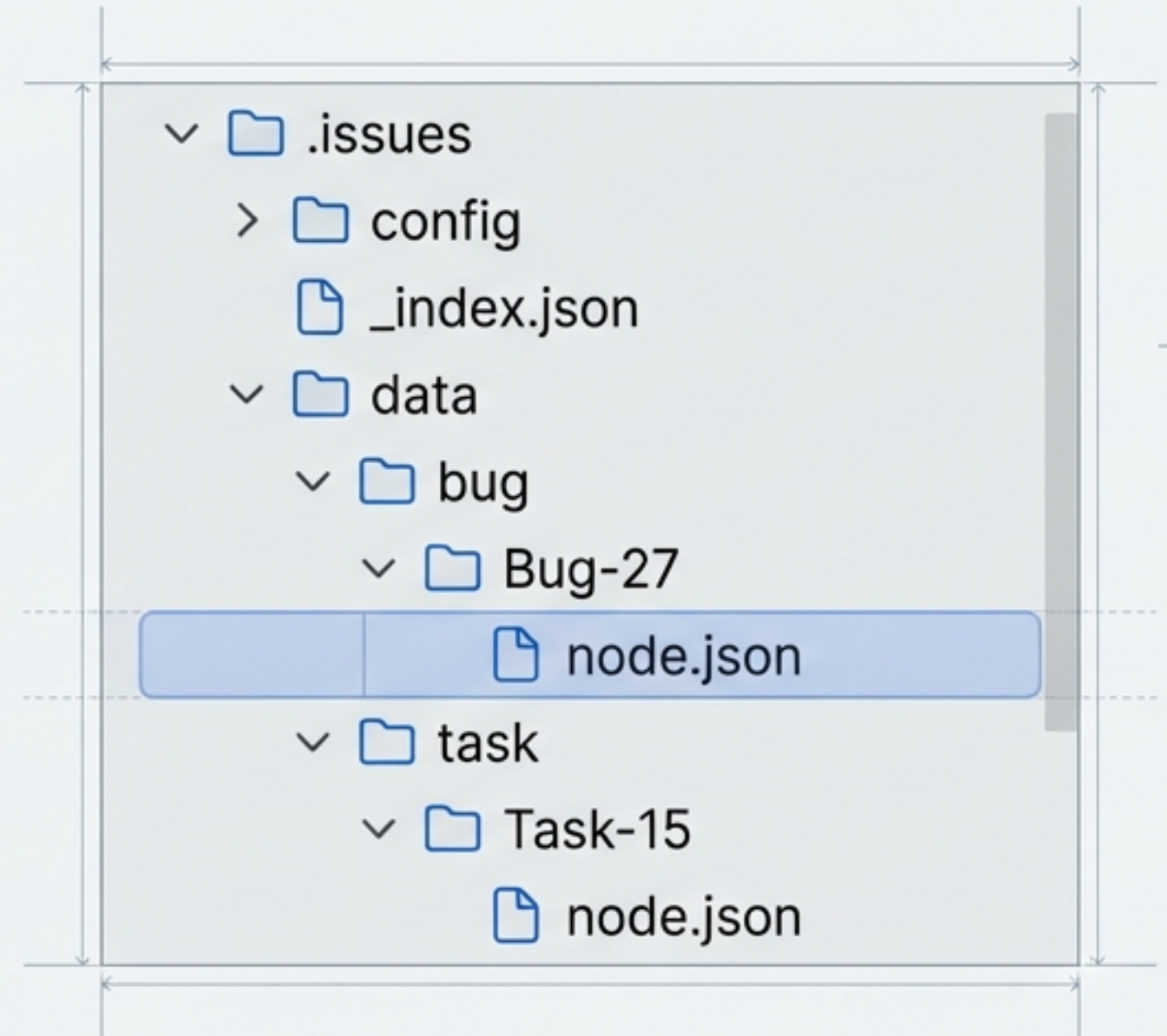
Definitions

- 1. Node ID (Machine):** A random 10-character alphanumeric GUID. Used for internal linking.
- 2. Node Index (Sequential):** A sequential counter specific to the node type.
- 3. Label (Human):** The derived identifier used for file naming and URL navigation.

Directory Structure & File System

The architecture is reflected directly in the file system, making the database browsable by humans on GitHub.

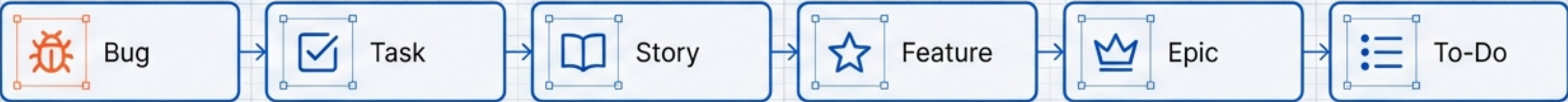
- **Folder per Issue:** Each node gets a dedicated folder. Allows for attachments and future extensions.
- **Type-Based Organisation:** Data is grouped logically (e.g., .issues/data/bug/).
- **Named by Label:** Directories use human-readable labels for immediate context.



Defining the Taxonomy (Node Types)

We track the entire project reality, not just code errors.

Work Items



Risk & Quality



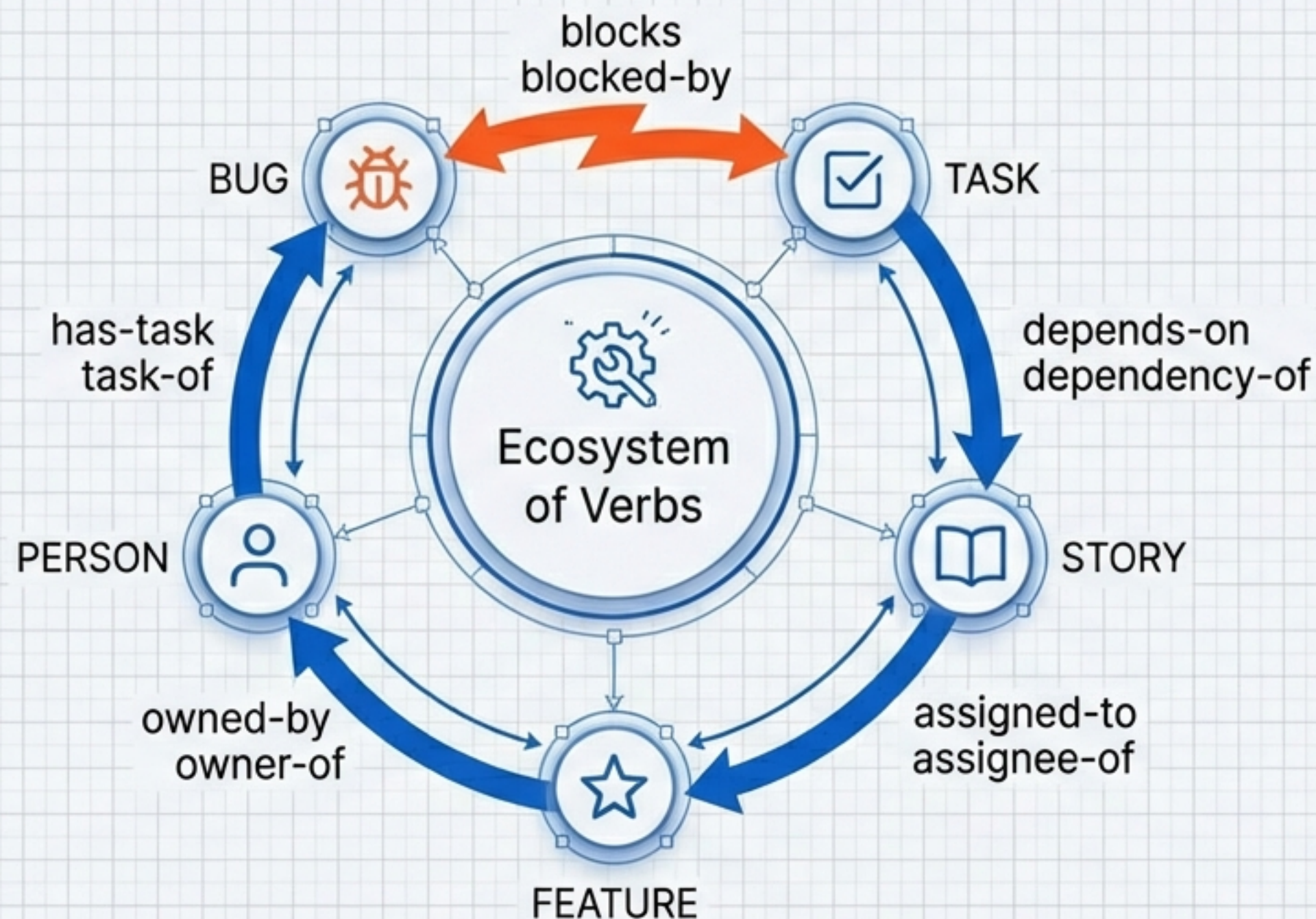
Meta & People



Each type tracks specific statuses (e.g., Vulnerabilities track reported -> verified; Risks track identified -> mitigated).

The Connective Tissue: Link Types

Relationships are bidirectional and denormalised. When Bug-27 blocks Task-15, both files are updated.



Key Categories

- **Workflow**
(blocks, depends-on)
- **Hierarchy**
(has-task, has-epic)
- **Ownership**
(assigned-to, owned-by)
- **Association**
(relates-to, duplicates)

The API Layer

A RESTful API provides structured access to the graph, handling node manipulation and traversal.

API Categories and Endpoints

Node Operations

GET /api/nodes/{label}

POST /api/nodes

PATCH /api/nodes/{label}

Graph Operations

GET /api/graph

GET /api/graph/subgraph/{label}

GET /api/graph/path/{from}/{to}

Link Operations

POST /api/nodes/{label}/links

API Card

GET /api/graph/subgraph/Bug-27

```
{
  "node": "Bug-27",
  "type": "bug",
  "status": "in-progress",
  "links": [
    { "verb": "blocks", "target": "Task-15" },
    { "verb": "assigned-to", "target": "Person-alice" }
  ]
}
```


Native AI Agent Integration

The system is explicitly designed for AI coordination, providing the structure agents need to reason about work.

Agent Enablers

- 1

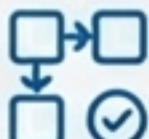
Structured Data: Strict JSON schemas prevent hallucination.


- 2

Clear Ownership: Person and Role nodes provide distinct identities.


- 3

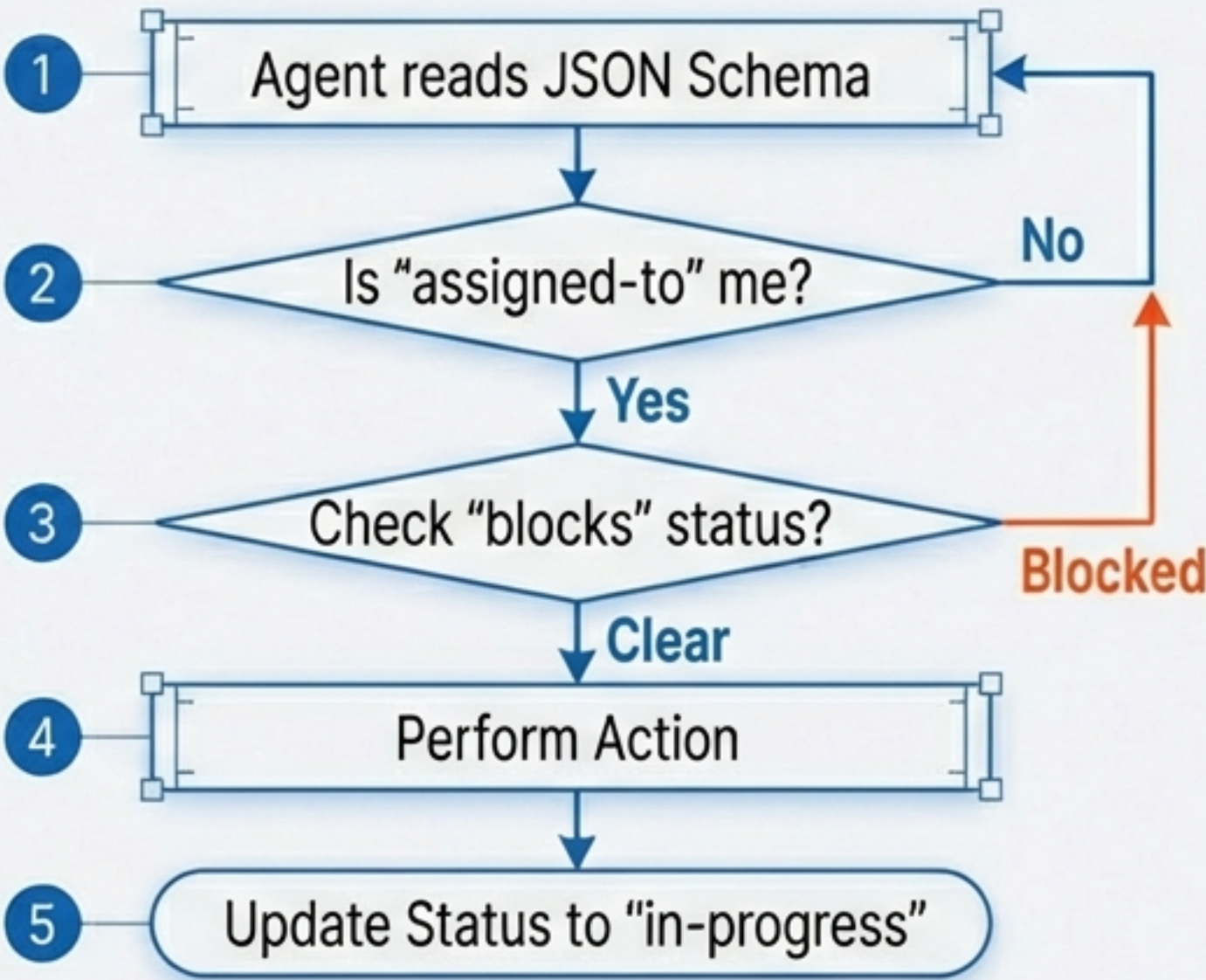
Dependency Tracking: Agents check "blocks" before acting.


- 4

Status Workflow: Agents can autonomously transition tasks.

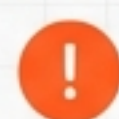


Agent Workflow



Implementation Strategy

Status: Architecture Design (v0.2.4)



WARNING: No Migration Path. This is a breaking change from v0.2.2.
The new structure requires a fresh start.

Phase 1: Core Infrastructure

Directory structure,
Types, CRUD

Phase 2: Relationships

Linking,
Validation

Phase 3: Attachments

Uploads, Image
Pipeline

Phase 4: Graph Ops

Path finding,
Export

Phase 5: Agents

Identity,
Workflows

Summary & Impact

Transforming a simple issue tracker into a complex, AI-coordinated project ecosystem.

Scales

Evolves to full project coordination.



Integrates

Native AI agent operation.



Persists

Git for absolute durability.



Visualises

Rich dependency mapping.

